audita

SMART CONTRACT SECURITY REVIEW · PREPARED FOR

OLY

An ERC-20 with dynamic sell taxes, time-locked staking, and Uniswap v4 liquidity defense

DATE
August 31, 2026

COMMIT
13e79ad

REMEDIATION
c8ca6e4

01 ABOUT

Audita

Audita is a Web3 security company specializing in smart contract security reviews for protocols, decentralized applications, and on-chain infrastructure.

We help teams identify vulnerabilities before they reach production through rigorous manual code review, adversarial security research, architecture analysis, and automated testing.

Our team consists of 50+ senior security researchers with proven experience across DeFi, infrastructure, wallets, trading systems, and complex on-chain applications.

To date, Audita has completed more than 100 security reviews across the Web3 ecosystem, working with teams at different stages ranging from pre-launch protocols to production systems securing significant on-chain value.

Every engagement is approached from both an offensive and defensive perspective: understanding how the system is intended to behave, identifying where those assumptions can break, and determining how an attacker could exploit them.

PREVIOUS WORK	WEBSITE
Audita public reports	audita.io

02 METHODOLOGY

How this review works

Researchers inspect the agreed source scope and audited commit, trace asset flows and privileged actions, challenge trust assumptions, and reproduce credible attack paths with targeted tests where feasible.

Each accepted finding records its impact and recommended remediation. We then review the remediation commit and record the outcome as Fixed, Partly fixed, or Acknowledged for that finding.

Risk classification

Severity reflects both the impact of a successful exploit and the likelihood of reaching it under the stated assumptions.

Likelihood / Impact	High	Medium	Low
High	HIGH	HIGH	MEDIUM
Medium	HIGH	MEDIUM	LOW
Low	MEDIUM	LOW	LOW

Informational findings have no direct security impact and sit outside the matrix.

IMPACT

High	Major asset loss, critical control compromise, or protocol-wide disruption.
Medium	Material but bounded loss or disruption.
Low	Limited loss, disruption, or scope.

LIKELIHOOD

High	Practical under ordinary on-chain conditions.
Medium	Requires specific but plausible conditions.
Low	Requires unusual conditions or disproportionate resources.

DISCLAIMER

A security review is bounded by time, resources, expertise, and the stated scope. It can identify vulnerabilities but cannot guarantee their absence. Resolution statuses describe only the remediation commit reviewed here. Later code, configuration, or integrations may change the risk. Continued testing, monitoring, and independent review are recommended.

03 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	PARTIAL OR ACKNOWLEDGED
18	11	7

SEVERITY DISTRIBUTION

7 Medium	4 Low	7 Informational

CONTRACTS IN SCOPE

src/LiquidityDefense.sol	src/libs/OlympusXHookLib.sol
src/OlympusXHook.sol	src/TaxRouter.sol
src/libs/MarketCapLib.sol	src/LaunchManager.sol
src/OlympusX.sol	src/libs/SwapActionsLib.sol
src/oracle/TruncGeoOracle.sol	

TIMELINE · AUGUST 2026

Audit · 6 days		Remediation · 4 days	
KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
August 22, 2026	August 27, 2026	August 24, 2026	August 27, 2026

STARTING COMMIT HASH	13e79adcaf38c299c6aa8bbd6e5539b269985d36
FINAL COMMIT HASH	c8ca6e4d849b620d58f774bd33687eb65caf8446

04 PROJECT SUMMARY

What OLY is

OLY, audited under the OlympusX name, lets users mint OLY during launch, stake it for 88 to 1,776 days, and trade it in a Uniswap v4 pool. Dynamic sell taxes fund stakers, validators, farms, token burns, and liquidity defense.

This review covers the stacked tax-settings, referral-removal, and liquidity-defense changes. The new defense vault places ETH-backed OLY buy walls across a frozen price ladder, burns OLY acquired during sell-offs, and allows any caller to deploy or settle the ladder while governance retains setup and recovery controls.

SECURITY POSTURE

BEFORE REMEDIATION 13e79ad

Unbounded state controls safety-critical execution

M-01 · M-02 · L-02 · L-03 · I-05

The defense gate, sell-tax tier, and aggregate accounting views trusted state a caller could move or grow beyond practical limits. This could arm overpriced walls, block deployment, misprice tax, or make whole-vault totals too expensive to query.

Wall allocation and retirement diverge from policy

M-03 · M-04 · M-05 · M-06 · I-01 · I-03

The allocation basis mixed idle ETH, live positions, eligibility, and weights. Price movement, call order, early setup, renunciation, and zeroed weights could strand capital or keep walls live outside policy.

Transfer accounting and operational references drift

M-07 · L-01 · L-04 · I-02 · I-04 · I-06 · I-07

Incentives, burns, mint floors, and router splits used pre-execution amounts, allowing overpayment, reverts, or the wrong allocation. The deployment runbook and unused declarations also diverged from implemented behavior.

AFTER REMEDIATION c8ca6e4

11 of 18 findings fixed and verified. Remediation bounds oracle cardinality reads, preserves wall allocations, makes deployment order independent, corrects ladder rounding and transfer accounting, cancels retired positions, and aligns deployment documentation with the code. The wall gate received block-open, high-water, and release controls, but M-01 remains partly fixed pending final validation of the residual economic path.

The six acknowledged findings leave residual risk around early wall setup, ownership renunciation, sell-tax timing, zero-value router dispatch, pending-epoch view growth, and the long-idle oracle edge. The defense vault should remain conservatively funded until M-01 is closed and the acknowledged operational risks are bounded and monitored.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`src/LiquidityDefense.sol`	Ownable ETH vault that places standing OLY buy walls at a frozen price ladder, burns acquired OLY, and exposes permissionless deployment and settlement.
`src/OlympusXHook.sol`	UUPS Uniswap V4 hook that applies market-sell and limit-order taxes, maintains the truncated oracle, and settles limit orders. The owner controls upgrades, tax tiers, oracle settings, and trusted endpoints.
`src/TaxRouter.sol`	UUPS router that splits ETH among stakers, buy and burn, validators, farms, liquidity defense, and the genesis wallet. The owner controls recipients, allocation basis points, and upgrades.
`src/LaunchManager.sol`	UUPS launch coordinator that accepts ETH during mint, creates liquid and vested OLY positions, seeds liquidity, and routes launch proceeds. The owner starts and extends the mint phase and controls protocol bindings.

Diff audit of 624 nSLOC across nine files. LiquidityDefense.sol was reviewed in full, while existing contracts and libraries were limited to the stacked PR #111, #112, and #113 changes.

PRIVILEGED ROLES

DAO timelock. Intended owner of the protocol contracts. Controls upgrades, tax configuration, launch bindings, liquidity-defense weights, pauses, retreat, and emergency recovery.

Community governor and Archon. Propose operations through the timelock, with the community governor using staked voting power and the Archon providing the optimistic proposal path.

Proedros. Holds the timelock canceller role and can veto queued governance operations.

Tax setter whitelists. Approved operators can toggle hook taxation or mark token-transfer endpoints taxable within owner-set bounds.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	Wall funding gate is priced off the pool it defends	MEDIUM	PARTLY
M-02	Unpermissioned cardinality growth pushes the oracle scan past block gas	MEDIUM	FIXED
M-03	A depressed-price deploy strands the vault's ETH allocation	MEDIUM	FIXED
M-04	Zeroing a wall's weight leaves its funded position live	MEDIUM	FIXED
M-05	Caller-selected deploy order decides which walls receive scarce ETH	MEDIUM	FIXED
M-06	Every buy wall opens above its configured price	MEDIUM	FIXED
M-07	`buyNBurn` pays its incentive on ETH that was never swapped	MEDIUM	FIXED
L-01	Taxing the PoolManager reverts OLY fee burns in the hook	LOW	FIXED
L-02	Market sell tax is cheapest for the fastest dump	LOW	ACKNOWLEDGED
L-03	Oracle accumulator drops the high bits of the time gap	LOW	ACKNOWLEDGED
L-04	Unchecked `uint160` cast can wrap a derived square-root price	LOW	FIXED
I-01	Walls can be configured and locked before the vault is funded	INFORMATIONAL	ACKNOWLEDGED
I-02	Mint checks `minTokensOut` against the pre-tax amount	INFORMATIONAL	FIXED
I-03	Renouncing ownership can freeze idle vault ETH	INFORMATIONAL	ACKNOWLEDGED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
I-04	Zero-value `distributeEthFromMint` uses the mint allocation on router funds	INFORMATIONAL	ACKNOWLEDGED
I-05	Unbounded pending epochs make accounting views impractical	INFORMATIONAL	ACKNOWLEDGED
I-06	`DEPLOYMENTS.md` describes the wrong defense ladder	INFORMATIONAL	FIXED
I-07	Reviewed changes introduce unused declarations	INFORMATIONAL	FIXED

MEDIUM SEVERITY · 7 FINDINGS

M-01 MEDIUM PARTLY FIXED

Wall funding gate is priced off the pool it defends

`LiquidityDefense` funds ETH buy walls only when their tick sits above `gateTick`. The gate returns the higher of spot and a TWAP read from the same OLY pool the walls defend.

```
function gateTick() private view returns (int24) {
    (, int24 spotTick,,) = poolManager.getSlot0(poolId);
    ITruncGeoOracle oracle = ITruncGeoOracle(address(hook));
    uint32 oldest = V4OracleLib.getOldestObservationSecondsAgo(oracle, poolKey);
    if (oldest == 0) return spotTick;
    uint32 window = oldest < twapPeriod ? oldest : twapPeriod;
    (int24 twapTick,) = V4OracleLib.consult(oracle, poolKey, window);
    return spotTick > twapTick ? spotTick : twapTick;
}
```

ETH is `currency0`, so an attacker arms higher walls by pushing the tick down. After a crash the TWAP remains below spot in tick terms, so the maximum discards it and the gate collapses to caller-controlled spot. The attacker can buy OLY, call permissionless `deployLevels`, then sell into walls funded above the real market.

The first remediation added block-open observations and a release cap. A second gate change records the block maximum and scans the observed high tick across `shortTwapPeriod`. The committed regression tests show that same-block and one-block arming attempts release no ETH. A price held beyond the configured short window can still arm a wall, and the gate does not enforce a direct deviation limit against the longer mean. The release cap bounds exposure but does not by itself establish that the remaining path is uneconomic.

IMPACT

The measured original path took 35.48 ETH from a 950 ETH idle vault. The final remediation blocks the documented same-block and one-block reproductions, but the cost and maximum loss of a dislocation held beyond `shortTwapPeriod` require final validation before the finding can be closed.

RECOMMENDATION

Reject wall funding when spot sits too far below a longer-window mean. Treat the deviation as a circuit breaker rather than taking the maximum of lagging readings, and require recovery across a time window that a single caller cannot stamp.

TEAM

Tracked in audit issue #4. Addressed by commits `3876450` and `304e149` in PR #114. The report retains Partly fixed pending final validation against `c8ca6e4`.

M-02 MEDIUM FIXED

Unpermissioned cardinality growth pushes the oracle scan past block gas

`increaseCardinalityNext` is external, permissionless, and accepts any `uint16` target. Growing the ring initializes storage but leaves the new observations empty until a write at the old boundary promotes the full target.

`getOldestObservationSecondsAgo` then scans empty slots one by one:

```
if (!oldestObservation.initialized) {
    for (uint16 i = 1; i < state.cardinality; i++) {
        uint16 index = (oldestIndex + i) % state.cardinality;
        oldestObservation = _truncGeoOracle.getObservation(_poolKey, index);
        if (oldestObservation.initialized) {
            break;
        }
    }
}
```

An attacker can grow cardinality to `65_535`, wait for the current ring boundary, and trigger one outside pool write. The next defense or farm call scans about 65,000 empty slots. Measured cost exceeds one billion gas for `deploy`, while farm withdrawal paths become unreachable within the block limit.

IMPACT

Idle defense ETH cannot reach the walls, and the main Uniswap V4 farm position cannot be withdrawn through its only public path. The attacker pays the storage-growth cost once and can leave safety-critical operations permanently over the block gas limit.

RECOMMENDATION

When the calculated oldest slot is uninitialized, read slot zero directly because the ring has not wrapped. Cap cardinality at an operationally tested value, and test the worst-case read and write paths against the live chain gas limit.

TEAM

Reported in audit issue #3. Fixed in commit `a2c0a9e` as part of PR #114. The change replaces the sparse linear scan with a slot-zero fallback and caps cardinality growth.

M-03 MEDIUM FIXED

A depressed-price deploy strands the vault's ETH allocation

`_deploy` starts `deployable` with the whole idle balance but adds weight and live ETH only for levels that pass the current gate.

```
uint256 totalWeight;
uint256 deployable = address(this).balance;
for (uint256 i = 0; i < n; ++i) {
    if (!_isPlaceable(i, gate)) continue;
    totalWeight += levelWeight[i];
    deployable += _levelLiveEth(i);
}
```

When price falls through upper walls, their weights disappear from the denominator while their share of idle ETH remains in the numerator. The eligible walls receive more than their configured share. After price recovers, those overfunded walls are skipped rather than reduced, so the omitted levels cannot recover their allocation.

IMPACT

Permissionless `deploy` can permanently move the budget away from the first walls needed in a recovery. In the measured 100 ETH case, the top two walls lost 10.52 ETH of intended capacity to deeper walls, and later calls could not rebalance it.

RECOMMENDATION

Calculate the basis over every level with nonzero weight, whether currently placeable or not. Keep the unplaceable share idle and apply the gate only when placing each level.

```
for (uint256 i = 0; i < n; ++i) {
-    if (!_isPlaceable(i, gate)) continue;
+    if (levelWeight[i] == 0) continue;
    totalWeight += levelWeight[i];
    deployable += _levelLiveEth(i);
}
```

TEAM

Reported in audit issue #5. Fixed in commit `622cc8a` as part of PR #114. The allocation basis now includes every active weighted level while the gate is applied only to placement.

M-04 MEDIUM FIXED

Zeroing a wall's weight leaves its funded position live

`setLevelWeights` originally replaced configuration without cancelling existing hook positions.

```
function setLevelWeights(uint128[] calldata weights) external onlyOwner {
    if (!isSetUp) revert NotSetUp();
    require(weights.length == levels(), WeightsLengthMismatch());
    for (uint256 i = 0; i < levels(); ++i) {
        levelWeight[i] = weights[i];
    }
}
```

A zero weight blocks future funding but does not call `hook.kill`. The wall remains live at its frozen tick and can still fill above current policy. `renounceOwnership` checked only whether any positive weight remained, so governance could seal the vault while a nominally retired wall still held ETH.

IMPACT

Capital continues buying at a price governance explicitly removed. Once ownership is renounced, no caller can invoke the owner-only retreat path, leaving the position live until it fills.

RECOMMENDATION

Cancel or reduce any live wall that exceeds its new cap during `setLevelWeights`. Refuse ownership renunciation while a retired level retains live ETH. The remediation implements this lifecycle coupling.

TEAM

Reported in audit issue #7. Fixed across commits `5ae3145` and `938d6d8` in PR #114. Resizing now cancels positions over their new cap and renunciation refuses an over-cap wall.

M-05 MEDIUM FIXED

Caller-selected deploy order decides which walls receive scarce ETH

`deployLevels` lets any caller select indices and processing order:

```
function deployLevels(uint256[] calldata indices) external {
    _deploy(indices);
}
```

Targets are calculated from a basis that includes live ETH in every placeable level. A level already above target is skipped, but its surplus remains inside `deployable`. Under-target levels then request more ETH than the idle balance holds, and each request is clipped to the balance remaining at that point in the caller's array.

IMPACT

The first permissionless caller chooses which walls receive scarce ETH. In the measured example, reversing the selected order funded four deep walls and left three nearer walls empty, changing OLY burned during the same drawdown by 3.70 times.

RECOMMENDATION

Remove above-target levels from the allocation basis and recompute until every included level is at or below target. The remaining deficits will then sum to the idle balance, making call order irrelevant.

TEAM

Reported in audit issue #10. Fixed in commit `77651da` as part of PR #114. The allocation basis now removes saturated levels before distributing scarce idle ETH.

M-06 MEDIUM FIXED

Every buy wall opens above its configured price

The pool uses ETH as `currency0` and OLY as `currency1`, so lower ticks represent higher OLY prices in USD. `_tickForOlyPriceUsd` rounds a positive reciprocal tick down to spacing:

```
int24 compressed = raw / spacing;
if (raw < 0 && raw % spacing != 0) compressed--;
return compressed * spacing;
```

Every configured ladder tick in the shipped setup is positive. Truncation therefore places each wall one band below the exact tick, which opens it above the intended USD price. Setup freezes these ticks permanently.

IMPACT

All eleven walls buy OLY above their configured ceiling. At the tested ETH price and 95 ETH ladder, rounding up would buy about 4.40 million more OLY than the deployed calculation, a 198 bps difference.

RECOMMENDATION

Round a positive reciprocal tick up so the funded band begins at or below the configured OLY price.

```
int24 compressed = raw / spacing;
-if (raw < 0 && raw % spacing != 0) compressed--;
+if (raw > 0 && raw % spacing != 0) compressed++;
return compressed * spacing;
```

TEAM

Reported in audit issue #6. Fixed in commit `453f16a` as part of PR #114. Positive reciprocal ticks now round up to spacing.

M-07 MEDIUM FIXED

`buyNBurn` pays its incentive on ETH that was never swapped

`buyNBurn` calculates the caller incentive from the capped balance before executing the swap. The retry loop can shrink the swap to 75% repeatedly when slippage checks fail, and returns the difference as `result.unusedAmount`.

```
uint128 incentive = _calculateIncentive(balance);
uint128 amountToSwap = uint128(balance) - incentive;
SwapActions.SwapExactInResult memory result = _swapV4Currency(
    SwapActions.SwapExactInParams({
        amountIn: amountToSwap,
        slippage: slippageBps,
        twapPeriod: twapPeriod
    }),
    key,
    Currency.wrap(address(0)),
    Currency.wrap(address(olympusX)),
    _deadline
);
_safeETHTransfer(msg.sender, incentive);
```

The payout ignored `unusedAmount`, so a shallow pool let the caller receive the configured percentage of a much larger requested swap. At a 5 ETH cap, only about 1.57 ETH executed while the caller still received 0.025 ETH.

IMPACT

Callers earn more than the configured incentive rate, and the overpayment grows as governance raises `swapCap` beyond available pool depth.

RECOMMENDATION

Scale the incentive by the fraction of requested input consumed. The remediation applies this calculation before payout.

```
+uint256 consumed = uint256(amountToSwap) - result.unusedAmount;
+incentive = uint128(uint256(incentive).mulDiv(consumed, amountToSwap));
_safeETHTransfer(msg.sender, incentive);
```

TEAM

Reported in audit issue #14. Fixed in commit `31cd0a1` as part of PR #114. The incentive is now scaled to the swap input actually consumed.

LOW SEVERITY · 4 FINDINGS

L-01 LOW FIXED

Taxing the PoolManager reverts OLY fee burns in the hook

`handleLiquidityFees` moved `amount1Fee` OLY from the PoolManager to the hook, then burned the same pre-transfer amount.

```
poolManager.mint(hookAddress, key.currency1.toId(), amount1Fee);
poolManager.burn(hookAddress, key.currency1.toId(), amount1Fee);
key.currency1.take(poolManager, hookAddress, amount1Fee, false);
ERC20Burnable(Currency.unwrap(key.currency1)).burn(amount1Fee);
```

OLY charges transfer tax when either endpoint is marked taxable. If governance or a transfer-tax setter marks the singleton PoolManager taxable, the hook receives only the net amount and `burn(amount1Fee)` reverts for insufficient balance. The path runs during order placement, cancellation, and fill, so a filled order can revert the entire swap.

IMPACT

Tax configuration for another OLY pool can disable limit-order operations and revert ordinary swaps that fill an order in the main pool.

RECOMMENDATION

Measure the hook balance before and after `take`, then burn and account for only the received delta. The remediation implements this balance-delta approach.

TEAM

Reported in audit issue #18. Fixed in commit `5f6152b` as part of PR #114. The hook now burns the measured OLY balance delta received from the PoolManager.

L-02 LOW ACKNOWLEDGED

Market sell tax is cheapest for the fastest dump

The hook writes at most one observation per block using the pre-swap tick. A market sell therefore resolves its dynamic tax tier from a TWAP that cannot include the swap being taxed.

```
function _beforeSwap(PoolKey calldata key) internal {
    _updatePool(key);
}
```

When `secondsAgo` is zero, the oracle returns the observation already stored for that block. A large single-block dump pays the tier selected before it moves price, while tranches separated across blocks progressively pay the higher tiers created by earlier sales.

IMPACT

The tax schedule rewards selling quickly. In the measured one-million OLY example, a single-block dump paid 599 bps while four delayed tranches paid 742 bps, leaving the fast seller about 0.0482 ETH more.

RECOMMENDATION

Use the TWAP as a stable starting price, then adjust market cap by the current swap's own price impact before selecting the tier. Apply the same calculation to exact-input and exact-output charge sites.

TEAM

Tracked in audit issue #15. No code remediation for this finding was included in PR #114, so the report retains Acknowledged.

L-03 LOW ACKNOWLEDGED

Oracle accumulator drops the high bits of the time gap

`TruncatedOracle.transform` shifted `delta` while it was still a `uint144`:

```
secondsPerLiquidityCumulativeX128: last.secondsPerLiquidityCumulativeX128
    + ((uint144(delta) << 128) / (liquidity > 0 ? liquidity : 1)),
```

Only 16 bits remain above the shift, so the accumulator keeps `delta % 65_536`. When the latest observation is older than the query window and a multiple of 65,536 seconds lies between the two transformed endpoints, the newer cumulative value becomes smaller. Checked subtraction in `V4OracleLib.consult` then reverts.

The condition recurs after more than 18.2 hours without a swap or liquidity change and lasts for the query-window duration. A fresh pool touch clears it.

IMPACT

`deploy`, defense views, and market-cap getters can revert during the window. The market sell path catches the failure and falls back to the top tax tier, so the issue does not disable selling.

RECOMMENDATION

Cast `delta` to `uint160` before shifting, matching the upstream accumulator width. The final remediation commit leaves the narrowing shift unchanged, so this finding remains acknowledged.

TEAM

Tracked in audit issue #19. No arithmetic fix for this finding was included in PR #114, so the report retains Acknowledged.

L-04 LOW FIXED

Unchecked `uint160` cast can wrap a derived square-root price

`_tickForOlyPriceUsd` originally narrowed the full Q64.96 square-root price with an unchecked cast:

```
uint160 sqrtPriceX96 = uint160(
    FullMath.mulDiv(sqrtScaled, 1 << 96, 1e18)
);
```

A value at or above `2**160` wraps before `TickMath.getTickAtSqrtPrice` validates it. Setup could then freeze a level at a tick unrelated to the configured USD ladder. The required ETH/USD input is unrealistic, but the constructor placed no upper bound on it and setup is one-shot.

IMPACT

An extreme or corrupted price input can permanently freeze the defense ladder at unintended ticks instead of reverting during setup.

RECOMMENDATION

Use the existing `SafeCast` helper so an out-of-range derived price reverts. The remediation applies `toUint160` before the tick conversion.

```
-uint160 sqrtPriceX96 = uint160(FullMath.mulDiv(sqrtScaled, 1 << 96, 1e18));
+uint160 sqrtPriceX96 = FullMath.mulDiv(sqrtScaled, 1 << 96, 1e18).toUint160();
```

TEAM

Fixed in commit `4d97572` as part of PR #114. The derived price now uses the existing checked `toUint160` conversion.

INFORMATIONAL SEVERITY · 7 FINDINGS

I-01 INFORMATIONAL ACKNOWLEDGED

Walls can be configured and locked before the vault is funded

`LiquidityDefense` receives only mint proceeds left after `initialLiquidityEth` is filled. Governance can call `setupLiquidityDefense` while the vault holds little ETH, and the one-time `levelWeight` values become absolute wei caps based on that early balance.

```
uint256 target = FullMath.mulDiv(deployable, levelWeight[i], totalWeight);
uint256 weight = levelWeight[i];
if (target > weight) target = weight;
```

Later mint proceeds can raise the vault balance far above the frozen caps. Once governance renounces ownership, `setLevelWeights`, `retreat`, and `escapeHatch` are unavailable, so the excess ETH can remain permanently unplaceable.

IMPACT

An early setup and renunciation sequence can strand most ETH delivered later in the mint, leaving it unable to support the configured defense ladder.

RECOMMENDATION

Represent level weights as proportions rather than absolute wei caps, and require the intended endowment before setup or ownership renunciation. Compute every level against one balance snapshot so the capacity tracks the funded vault.

TEAM

Acknowledged in audit issue #11. No code remediation for the one-shot absolute caps was included in PR #114.

I-02 INFORMATIONAL FIXED

Mint checks `minTokensOut` against the pre-tax amount

`LaunchManager.mint` checks the buyer's floor against gross `tokensToMint` before transferring OLY.

```
require(tokensToMint > 0, LaunchManager__VestingScheduleTotalTokensExceeded());
require(tokensToMint >= minTokensOut, LaunchManager__InsufficientTokensOut());
```

If either transfer endpoint is taxable, `OlympusX._update` sends only the net amount. A liquid buyer can receive less than the signed floor after the check passes. A staked schedule can revert because the staking vault is asked to deposit the gross amount even though it received less.

IMPACT

Transfer-tax configuration can make the mint floor ineffective for liquid schedules and can make taxed staked schedules unmintable.

RECOMMENDATION

Measure the OLY received by the buyer and staking vault, enforce `minTokensOut` against the measured total, and scale the staking amounts to the vault's actual receipt.

TEAM

Reported in audit issue #9. Fixed in commit `375e3bd` as part of PR #114. Mint delivery is measured before enforcing `minTokensOut`, and staked amounts use the tokens actually received.

I-03 INFORMATIONAL ACKNOWLEDGED

Renouncing ownership can freeze idle vault ETH

`renounceOwnership` checks that at least one wall has a nonzero weight, but it does not check the vault balance or whether all ETH can be placed.

```
bool anyActive;
for (uint256 i = 0; i < levels(); ++i) {
    if (levelWeight[i] > 0) {
        anyActive = true;
        break;
    }
}
require(anyActive, AllWallsRetired());
```

ETH above the summed wall capacity is idle. Renunciation permanently disables the owner-only recovery path, while later direct or mint inflows can increase the stranded balance.

IMPACT

Valid governance operations can leave ETH in the defense vault with no path to a wall or recovery recipient.

RECOMMENDATION

Add a permissionless post-renunciation release to the immutable buy-and-burn sink. Reserve the ladder's remaining capacity from frozen wall state before allowing any excess to leave.

TEAM

Acknowledged in audit issue #12. No post-renunciation release path was included in PR #114.

I-04 INFORMATIONAL ACKNOWLEDGED

Zero-value `distributeEthFromMint` uses the mint allocation on router funds

`_distributeEthInternal` falls back to the router's distributable balance when `msg.value` is zero. That fallback is intended for the manual distribution entry, but `distributeEthFromMint` exposes it with the mint allocation and no caller restriction.

```
function distributeEthFromMint() external payable nonReentrant {
    _distributeEthInternal(buyAndBurnMintBps, true);
}
```

Any caller can submit a zero-value call when the router holds distributable ETH and route that balance through the mint split, including the one-time liquidity-defense allocation.

IMPACT

Unexpected ETH held by the router can be distributed through allocation basis points intended only for mint proceeds.

RECOMMENDATION

Reject zero-value calls on `distributeEthFromMint`, leaving balance sweeping exclusively to the manual distribution entry point.

TEAM

Acknowledged in audit issue #16. The zero-value mint distribution path remains unchanged in PR #114.

I-05 INFORMATIONAL ACKNOWLEDGED

Unbounded pending epochs make accounting views impractical

Each filled wall can append a new entry to `pendingEpochs`, and only an unrewarded sweep removes entries. `unsettled` walks the entire array and makes external reads for each element.

```
for (uint256 i = 0; i < pendingEpochs.length; ++i) {
    (uint256 ethOwed, uint256 olyOwed,, bool filled) = _owedOf(Epoch.wrap(pendingEpochs[i]));
    if (!filled) continue;
    ethAmount += ethOwed;
    olyAmount += olyOwed;
}
```

Repeated fills can make `unsettled` and the `holdings` view that calls it exceed normal `eth_call` gas caps. Settlement itself remains available, but a single-call whole-vault total becomes unavailable.

IMPACT

Operators and integrations can lose reliable access to aggregate vault accounting as historical filled epochs accumulate.

RECOMMENDATION

Settle a bounded number of old epochs during ordinary deployment, and expose paginated pending-epoch accounting so every read has a fixed gas bound.

TEAM

Tracked in audit issue #13. The permissionless sweep is bounded, but the aggregate `unsettled` and `holdings` views remain unbounded in PR #114, so the report retains Acknowledged.

I-06 INFORMATIONAL FIXED

`DEPLOYMENTS.md` describes the wrong defense ladder

The deployment runbook says `setupLiquidityDefense` uses nine levels from $0.001 to $0.01. The deployment code constructs eleven levels and adds a $0.0008 bedrock wall.

```
p = new uint256[](11);
p[0] = 0.0008e18;
```

Following the documented nine-weight example causes `WeightsLengthMismatch` during the one-time setup call.

IMPACT

An operator following the runbook cannot complete defense setup and may prepare an allocation that omits two deployed levels.

RECOMMENDATION

Document the eleven-level $0.0008 to $0.01 ladder, or derive the count and range directly from `defenseLadderUsd18` in the deployment procedure.

TEAM

Reported in audit issue #17. Fixed in commit `ec5b542` as part of PR #114. `DEPLOYMENTS.md` now documents eleven levels from $0.0008 to $0.01.

I-07 INFORMATIONAL FIXED

Reviewed changes introduce unused declarations

The reviewed changes add `NothingToEscape` and `MARKET_SELL_TAX_BPS`, but no execution path references either declaration.

```
error NothingToEscape();
uint16 public constant MARKET_SELL_TAX_BPS = 1000;
```

IMPACT

Unused declarations add noise to the public code surface and can mislead maintainers about behavior that the contracts enforce.

RECOMMENDATION

Remove the unused error and constant until an implemented path needs them.

TEAM

Reported in audit issue #2. Fixed in commit `8ed422b` as part of PR #114. The unused declarations were removed.